

lab2.py

```
from src import imaging, linalg, plotting
```

```
ks = [5, 20, 50]
```

```
# Task 1: square image - EVD + SVD
```

```
img = imaging.load_image("lab-2-img/cat_02_square.png")
```

```
img = imaging.resize(img, (100, 100))
```

```
A = imaging.to_grayscale(img)
```

```
eigenvalues, Q = linalg.evd(A)
```

```
U, s, Vt = linalg.svd(A)
```

```
for k in ks:
```

```
    a_k_evd = linalg.reconstruct_evd(eigenvalues, Q, k)
```

```
    a_k_svd = linalg.reconstruct_svd(U, s, Vt, k)
```

```
    print(f"square k={k} E_evd={linalg.frobenius_norm(A - a_k_evd):.2f} E_svd={linalg.frobenius_norm(A - a_k_svd):.2f}")
```

```
    plotting.show_reconstruction(A, a_k_evd, title=f"square EVD, k={k}")
```

```
    plotting.show_reconstruction(A, a_k_svd, title=f"square SVD, k={k}")
```

```
n = len(eigenvalues)
```

```
all_ks = list(range(1, n + 1))
```

```
errors_evd = [linalg.frobenius_norm(A - linalg.reconstruct_evd(eigenvalues, Q, k)) for k in all_ks]
```

```
errors_svd = [linalg.frobenius_norm(A - linalg.reconstruct_svd(U, s, Vt, k)) for k in all_ks]
```

```
plotting.error_curve(all_ks, {"evd": errors_evd, "svd": errors_svd}, title="square image: E(k) vs k")
```

```
# Task 2: rectangular image - SVD only
```

```
img2 = imaging.load_image("lab-2-img/cat_02_rectangle.png")
```

```
img2 = imaging.resize_preserve_aspect(img2, 100)
```

```
A2 = imaging.to_grayscale(img2)
```

```
U2, s2, Vt2 = linalg.svd(A2)
```

```
for k in ks:
```

```
    a_k = linalg.reconstruct_svd(U2, s2, Vt2, k)
```

```
    print(f"rectangle k={k} E_svd={linalg.frobenius_norm(A2 - a_k):.2f}")
```

```
    plotting.show_reconstruction(A2, a_k, title=f"rectangle SVD, k={k}")
```

```
n2 = len(s2)
```

```
all_ks2 = list(range(1, n2 + 1))
```

```
errors2 = [linalg.frobenius_norm(A2 - linalg.reconstruct_svd(U2, s2, Vt2, k)) for k in all_ks2]
```

```
plotting.error_curve(all_ks2, {"svd": errors2}, title="rectangle image: E(k) vs k")
```

```

# src/imaging.py

import matplotlib.image as mpimg
import numpy as np
from PIL import Image

# load a PNG as an (H, W, 3) uint8 RGB array
def load_image(path):
    img = mpimg.imread(path)
    if img.dtype != np.uint8:
        img = (img * 255).astype(np.uint8)
    return img[..., :3]

# resize to (new_width, new_height)
def resize(img, size):
    pil_img = Image.fromarray(img)
    resized = pil_img.resize(size)
    return np.array(resized)

# resize so the longer side becomes max_side, keeping aspect ratio
def resize_preserve_aspect(img, max_side):
    h, w = img.shape[:2]
    if w >= h:
        new_w = max_side
        new_h = round(h * max_side / w)
    else:
        new_h = max_side
        new_w = round(w * max_side / h)
    return resize(img, (new_w, new_h))

# convert (H, W, 3) RGB to (H, W) grayscale
def to_grayscale(img):
    R = img[..., 0].astype(float)
    G = img[..., 1].astype(float)
    B = img[..., 2].astype(float)
    return 0.299 * R + 0.587 * G + 0.114 * B

```

```

# src/linalg.py (evd/svd functions used by lab2.py only)
import numpy as np

# eigendecomposition, sorted by descending eigenvalue magnitude
def evd(A):
    eigenvalues, Q = np.linalg.eig(A)
    order = np.argsort(np.abs(eigenvalues))[:, :-1]
    return eigenvalues[order], Q[:, order]

# rank-k reconstruction  $A_k = Q * \lambda_k * Q^{-1}$ 
def reconstruct_evd(eigenvalues, Q, k):
    n = len(eigenvalues)

    # keep conjugate pairs together
    if 0 < k < n and np.isclose(eigenvalues[k], np.conj(eigenvalues[k - 1])):
        k += 1

    lambda_k = np.zeros(n, dtype=complex)
    lambda_k[:k] = eigenvalues[:k]
    lambda_k = np.diag(lambda_k)

    q_inv = np.linalg.inv(Q)
    a_k = Q @ lambda_k @ q_inv
    return np.real(a_k)

# singular value decomposition, sorted descending by construction
def svd(A):
    U, singular_values, Vt = np.linalg.svd(A)
    return U, singular_values, Vt

# rank-k reconstruction  $A_k = U * \sigma_k * V^t$ 
def reconstruct_svd(U, singular_values, Vt, k):
    m = U.shape[0]
    n = Vt.shape[0]

    sigma_k = np.zeros((m, n))
    sigma_k[:k, :k] = np.diag(singular_values[:k])

    return U @ sigma_k @ Vt

# frobenius norm: sqrt of sum of squares of every entry
def frobenius_norm(A):
    return np.sqrt(np.sum(A ** 2))

```

```

# src/plotting.py (functions used by lab2.py only)

import matplotlib.pyplot as plt
import numpy as np

# show original, reconstructed and error image side by side
def show_reconstruction(original, reconstructed, title=""):
    error = np.abs(original - reconstructed)

    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    axes[0].imshow(original, cmap="gray", vmin=0, vmax=255)
    axes[0].set_title("original")
    axes[1].imshow(reconstructed, cmap="gray", vmin=0, vmax=255)
    axes[1].set_title("reconstructed")
    axes[2].imshow(error, cmap="gray")
    axes[2].set_title("error")

    for ax in axes:
        ax.axis("off")

    fig.suptitle(title)
    plt.show()

# plot E(k) vs k, series = {label: errors}
def error_curve(ks, series, title=""):
    plt.figure()
    for label, errors in series.items():
        plt.plot(ks, errors, label=label)
    plt.xlabel("k")
    plt.ylabel("E(k)")
    plt.title(title)
    plt.legend()
    plt.show()

```